LIMO Milestone M3 — Pipeline Core

Team Binary Brains · Late submission

Release tag	`m3` — link: _____
Repository	https://github.com/MAKHAOLA-N/LIMO_BINARY_BRAINS
Live site	_____
Run the tests	`node tests/run.js` → **137 passed, 0 failed**

Late-submission note. M3 was due 18 October. We are submitting it late. The `m3` tag points at our current code, which also contains the M4 work (forwarding, stalls, flushes and the live views). Everything M3 asks for is listed below with where to find it. The extra M4 parts do not change how a hazard-free program runs: section 3 shows it takes the same 14 cycles with forwarding ON or OFF.

1. M3 requirements and where they are

M3 asks for	Where on the page	Where in the code	
Five stages IF, ID, EX, MEM, WB	Panel 3 (datapath), "Stage now" column in panel 2	`step()` in `src/pipeline.js`	✓
Pipeline registers IF/ID, ID/EX, EX/MEM, MEM/WB	Panel 3, table "Contents of every pipeline register"	`src/pipeline-model.js`	✓
Control unit	Panel 4 (control signals)	`controlSignals()` in `src/pipeline.js`	✓
Separate instruction and data memories (B2)	Panel 2 (instructions), panel 9 (data)	`state.program` / `state.mem`	✓
Step, Run, Reset	Panel 3 toolbar (+ Pause, Step back, breakpoints)	`src/app.js`	✓
Register view	Panel 8: r0–r15 with Sesotho names	`renderRegisters()` in `src/views.js`	✓
Memory view	Panel 9: 256 bytes as 64 words	`renderMemory()` in `src/views.js`	✓
Hazard-free programs run correctly	Example **0-hazard-free**	`examples/0-hazard-free.limo`, `tests/programs.test.js`	✓

2. How the pipeline core works

One call to `step(state)` is **one clock cycle**. Inside it the stages run in the order **WB → MEM → EX → ID → IF**. Each stage reads the *old* pipeline register on its left and builds a *new* one on its right. At the end, all four new registers and the PC are saved together. That moment is the **clock edge**. WB runs first so the register file is written before ID reads it.

```
WB  : if RegWrite → regs[rd] = MemtoReg ? memData : aluResult
MEM : if MemRead → memData = mem[addr];  if MemWrite → mem[addr] = storeData
EX  : B = ALUSrc ? imm : rs2;  result = ALU(ALUOp, rs1, B);  branch target = PC+4 + imm×4
ID  : decode the word, read rs1 and rs2, sign-extend imm, controlSignals(name)
IF  : fetch program[PC/4];  PC = PC + 4
clock edge: IFID, IDEX, EXMEM, MEMWB, PC ← the new values
```

Control unit (made in ID, carried down the pipeline)

Instruction	RegWrite	ALUSrc	ALUOp	MemRead	MemWrite	MemtoReg	Branch
R-type (eketsa, fokotsa, kopanya …)	1	0	add/sub/and…	0	0	0	0
eketsal / kopanyal / kapal	1	1	add/and/or	0	0	0	0
jarisa (load)	1	1	add	1	0	1	0
ngola (store)	0	1	add	0	1	0	0
lekana / fapane	0	0	sub	0	0	0	1

All 15 rows are checked automatically in `tests/pipeline.test.js` (section P2).

3. Evidence: a hazard-free program runs correctly

What "hazard-free" means. No instruction reads a register less than 3 instructions after it was written. A result is written back in cycle p+5, and an instruction k lines later reads registers in ID in cycle k+2. WB happens before ID in the same cycle, so $k - p \geq 3$ is safe. Then the pipeline never has to stall, forward or flush.

```
.data 0  9
        eketsaI r1, r0, 7      # r1 = 7
        eketsaI r2, r0, 5      # r2 = 5
        jarisa  r6, 0(r0)      # r6 = 9
        eketsaI r7, r0, 1      # r7 = 1
        eketsa  r3, r1, r2     # r3 = 12
        fokotsa r4, r1, r2     # r4 = 2
        kopanya r5, r6, r1     # r5 = 1
        ngola   r3, 8(r0)      # mem[8] = 12
        lekana  r1, r2, end    # not taken
end:    eketsaI r8, r0, 3      # r8 = 3
```

	Expected (by hand)	Simulator
Registers	r1=7 r2=5 r3=12 r4=2 r5=1 r6=9 r7=1 r8=3	**same**
Memory	[0]=9, [8]=12	**same**
Cycles	10 instructions + 4 = 14	**14**
Stalls / flushes	0 / 0	**0 / 0**
Forwarding OFF	still 14 cycles	**14**
Hazards reported	none	**0 in all 14 cycles**

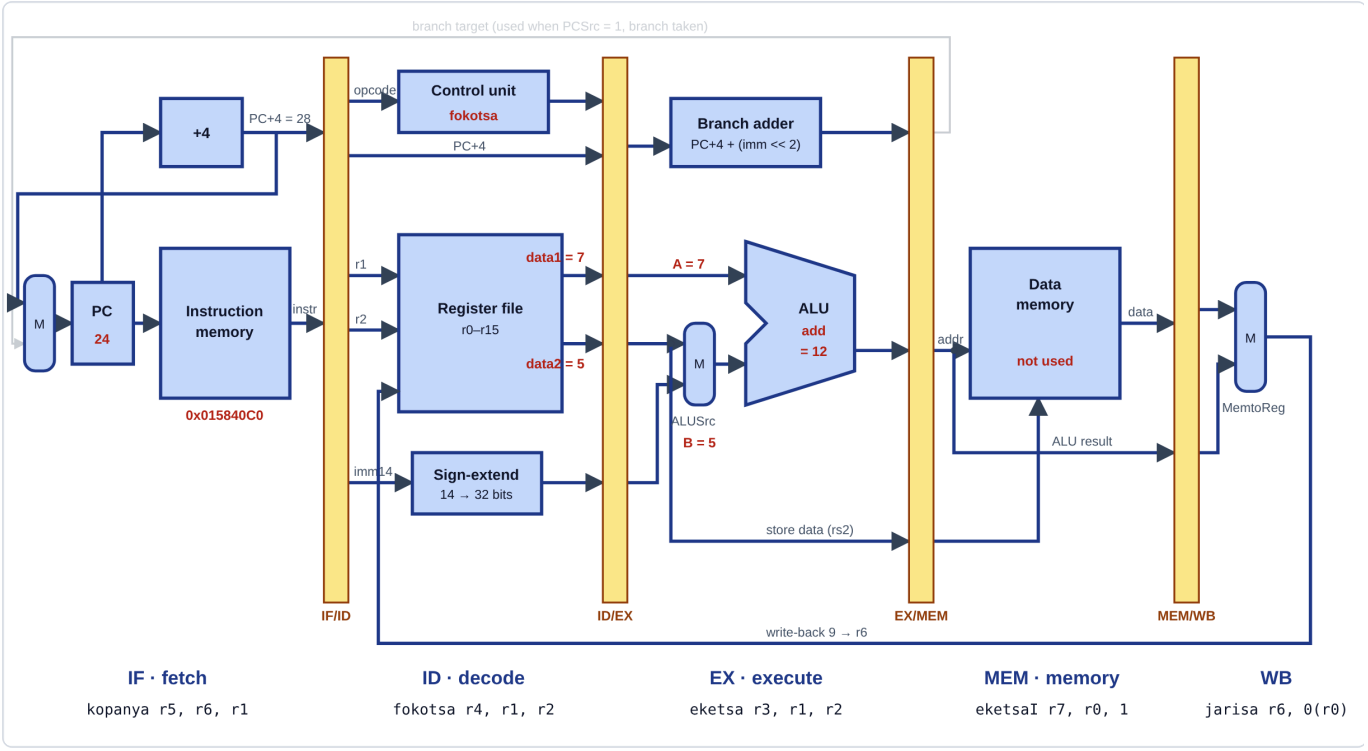
It covers every instruction class: arithmetic, immediate, logic, load, store and branch (not taken).

Cycle-by-cycle trace (taken from the simulator)

Cycle	IF	ID	EX	MEM	WB	Memory / register written
1	`eketsaI r1, r0, 7`	–	–	–	–	
2	`eketsaI r2, r0, 5`	`eketsaI r1, r0, 7`	–	–	–	
3	`jarisa r6, 0(r0)`	`eketsaI r2, r0, 5`	`eketsaI r1, r0, 7`	–	–	
4	`eketsaI r7, r0, 1`	`jarisa r6, 0(r0)`	`eketsaI r2, r0, 5`	`eketsaI r1, r0, 7`	–	
5	`eketsa r3, r1, r2`	`eketsaI r7, r0, 1`	`jarisa r6, 0(r0)`	`eketsaI r2, r0, 5`	`eketsaI r1, r0, 7`	**r1 ← 7**
6	`fokotsa r4, r1, r2`	`eketsa r3, r1, r2`	`eketsaI r7, r0, 1`	`jarisa r6, 0(r0)`	`eketsaI r2, r0, 5`	**read [0] = 9; r2 ← 5**
7	`kopanya r5, r6, r1`	`fokotsa r4, r1, r2`	`eketsa r3, r1, r2`	`eketsaI r7, r0, 1`	`jarisa r6, 0(r0)`	**r6 ← 9**
8	`ngola r3, 8(r0)`	`kopanya r5, r6, r1`	`fokotsa r4, r1, r2`	`eketsa r3, r1, r2`	`eketsaI r7, r0, 1`	**r7 ← 1**
9	`lekana r1, r2, end`	`ngola r3, 8(r0)`	`kopanya r5, r6, r1`	`fokotsa r4, r1, r2`	`eketsa r3, r1, r2`	**r3 ← 12**
10	`eketsaI r8, r0, 3`	`lekana r1, r2, end`	`ngola r3, 8(r0)`	`kopanya r5, r6, r1`	`fokotsa r4, r1, r2`	**r4 ← 2**
11	–	`eketsaI r8, r0, 3`	`lekana r1, r2, end`	`ngola r3, 8(r0)`	`kopanya r5, r6, r1`	**write [8] ← 12; r5 ← 1**
12	–	–	`eketsaI r8, r0, 3`	`lekana r1, r2, end`	`ngola r3, 8(r0)`	**–**
13	–	–	–	`eketsaI r8, r0, 3`	`lekana r1, r2, end`	**–**
14	–	–	–	–	`eketsaI r8, r0, 3`	**r8 ← 3**

Read a diagonal: `eketsaI r1` is in IF in cycle 1, ID in 2, EX in 3, MEM in 4 and WB in 5. One instruction finishes every cycle from cycle 5 onwards, which is the ideal CPI of 1 once the pipeline is full. `eketsa r3, r1, r2` reads r1 and r2 in cycle 6, after both were written back in cycles 5 and 6.

4. The program on the simulator



Cycle 7: all five stages are full. EX adds 7 + 5 = 12 for `eketsa r3`; WB writes 9 into r6 for `jarisa`; the PC is 24.

IF/ID	ID/EX	EX/MEM	MEM/WB
kopanya r5, r6, r1 pc = 24 pcPlus4 = 28 instruction = 0x015840C0	**fokotsa r4, r1, r2** pc = 20 pcPlus4 = 24 rs1 = 1 rs2 = 2 readsRs2 = true rd = 4 readData1 = 7 readData2 = 5 imm = 0 control: RegWrite, ALUOp sub	**eketsa r3, r1, r2** pc = 16 rd = 3 aluResult = 12 storeData = 5 branchTarget = 20 branchTaken = false control: RegWrite, ALUOp add	**eketsaI r7, r0, 1** pc = 12 rd = 7 aluResult = 1 memData = 0 control: RegWrite, ALUSrc, ALUOp add

Values saved at the end of cycle 7 (the clock edge).

The four pipeline registers at the end of cycle 7.

8. Register file *B6*

◀ = changed this cycle.

Reg	Sesotho name	Decimal	Hex
r0	lefela (zero)	0	0x00000000
r1	n1	7	0x00000007
r2	n2	5	0x00000005
r3	n3	12	0x0000000C
r4	n4	2	0x00000002
r5	n5	1	0x00000001
r6	n6	9	0x00000009
r7	n7	1	0x00000001
r8	**n8**	**3 ◀**	**0x00000003**
r9	n9	0	0x00000000
r10	n10	0	0x00000000
r11	n11	0	0x00000000
r12	n12	0	0x00000000
r13	n13	0	0x00000000
r14	mokokotlo (sp)	0	0x00000000

Register view after 14 cycles (r8 changed last, marked ◀).

9. Data memory *B6*

256 bytes, little-endian, shown as 64 words. ◀ = changed this cycle.

Address	Decimal	Hex
0	9	0x00000009
4	0	0x00000000
8	12	0x0000000C
12	0	0x00000000
16	0	0x00000000
20	0	0x00000000
24	0	0x00000000
28	0	0x00000000
32	0	0x00000000
36	0	0x00000000
40	0	0x00000000
44	0	0x00000000
48	0	0x00000000
52	0	0x00000000
56	0	0x00000000

Memory view: [0] = 9 from `.data`, [8] = 12 written by `ngola`.

5. How to demonstrate M3 (D2-style: trace a program cycle by cycle)

1. Open the site. The example **0-hazard-free** is loaded and assembled.
2. Press **Step** 7 times and stop. Point at the five stage names under the drawing: one instruction in each stage.
3. Explain EX: "data1 = 7, data2 = 5, ALUSrc = 0, so the ALU adds the two registers: 12."
4. Explain WB: "MemtoReg = 1 for `jarisa`, so the value read from memory, 9, goes into r6." Panel 8 shows r6 change.
5. Press **Run**. It finishes after 14 cycles. Show the register and memory views against the expected table.
6. Press **Reset**. Everything returns to cycle 0.

> Our pipeline runs one clock cycle per step. Each stage reads the old pipeline register on its left and writes a new one on its right, and all four are saved together at the clock edge. In a hazard-free program one instruction finishes every cycle once the pipeline is full, so 10 instructions take 10 + 4 = 14 cycles.

Questions to be ready for

Question	Answer
Why does a program of n instructions take n + 4 cycles?	The first instruction needs 5 cycles to reach WB. After that one more finishes every cycle.
Why do the stages run WB → IF inside `step()`?	So WB writes the register file before ID reads it, and each stage uses the old pipeline register, not one already overwritten this cycle.
Which signals are 1 for `ngola`?	ALUSrc (address = rs1 + offset) and MemWrite. RegWrite is 0 because a store writes memory, not a register.
Why is `eketsaI r7` in the program?	It is a filler. It puts 3 lines between `jarisa r6` and `kopanya` (which reads r6), so there is no load-use hazard.
Where is the clock edge in the code?	The last lines of `step()`, where `s.IFID`, `s.IDEX`, `s.EXMEM`, `s.MEMWB` and `s.pc` are replaced together.

Automated tests (one command: `node tests/run.js`)

Test file	Covers	Checks
`assembler.test.js`	Encodings, labels, `.data`, errors	41
`pipeline.test.js`	Decoder, control unit, ALU, hazard unit, memory	42
`programs.test.js`	9 programs incl. 0-hazard-free, forwarding ON and OFF	54
Total		**137 pass**

Sprint note — Milestone M3 (Pipeline core)

Sprint goal: a five-stage pipeline with pipeline registers and a control unit, Step/Run/Reset, register and memory views, and hazard-free programs running correctly.

Done

- Five-stage pipeline: one `step()` = one clock cycle (`src/pipeline.js`).
- Four pipeline registers as objects (`src/pipeline-model.js`); separate instruction and data memories.
- Control unit for all 15 instructions, checked by 15 automated tests.
- Step, Run, Reset (plus Pause, Step back and breakpoints).
- Register view with Sesotho names, and a 256-byte data memory view; changed values marked.
- Hazard-free test program `0-hazard-free.limo`: registers, memory and 14-cycle count match the hand-worked values.
- `.data` lines in the assembler, to put starting values in data memory.

Not done, and why

- The milestone was submitted late (due 18 Oct). Reason: _____
- Tag `m3` was not created on time, so it now points at code that also includes M4 work.

Next

- M5: Ripes cross-check (B8 iii), usability test with three first-year students (B8 iv), Sesotho narration checked by a speaker, user guide, final report, YouTube video, release v1.0.

Who did what

Member	Role this sprint	Delivered (issue / PR)	PRs reviewed
Makhaola Nkhahle			
Eric Letsoela			
Ramosamo Lekhela			
Hlabathe Moeko			
Kananelo Ketlele			
Bonolo Makhotla			
Ntoetse Selomo			

Suggested roles (handout §4): ISA architect, assembler lead, pipeline lead, hazard-unit lead, interface lead, test lead, scrum and release lead.

AI use this sprint

Claude was used to draft the pipeline core, views and tests. The team worked out the expected registers, memory and cycle count of each test program by hand before running the simulator, and each member traced the file they own (see `AI_USE.md`).